

0909 Baseline Report — Gemma-4-12B-it on FuriosaAI RNGD

작성일 2026-09-09 (최종 갱신 동일자) · 기준선 v0_baseline · 현재 SOTA v50_drop_false_replication (RNGD 실측 4.972x) 상위 규칙은 RULES.md, 실험 기록은 RESULTS.md, 신기록 서사는 SOTA.md에 있다.

0. 이 문서를 읽는 법

대상 독자. 학부 수준의 선형대수(행렬곱)와 C/Python 정도의 프로그래밍 경험이 있는 사람. NPU, 텐서 컴파일러, LLM 추론 최적화를 몰라도 읽을 수 있게 썼다. Rust를 몰라도 된다 — 이 문서에 나오는 Rust 코드는 "읽는 법"을 §3에서 따로 가르친다.

구성.

절	내용	이미 아는 사람은
§1	프로젝트와 대회가 뭔지	훑고 넘어가기
§2	NPU가 어떻게 동작하는가 (속성 과정)	건너뛰기
§3	furiosa-opt 코드 읽는 법	건너뛰기
§4	최적화 원시 카탈로그	여기가 핵심
§5~§8	대회 규칙, 코드 지도, 커널 해부, 측정 체계	필요할 때 찾아보기
§9	지금까지 뭘 했고 다음은 뭔지	여기가 핵심

§2~§4가 이번 판에서 새로 들어간 부분이다. 원리를 모르면 §9의 실험 목록이 그냥 주문처럼 보이기 때문에 넣었다.

1. 한눈에 보기

1.1 무엇을 하는 프로젝트인가

Google의 오픈 LLM **Gemma-4-12B-it**(120억 파라미터)을 **FuriosaAI RNGD**라는 AI 전용 칩 위에서 최대한 빠르게 돌리는 것이다. 이 저장소에는 이미 동작하는 구현이 통째로 들어 있다 — 모델 실행, 이미지·오디오 입력, HTTP 서버까지. 우리가 하는 일은 **그중 딱 세 개의 함수를 더 빠르게 고쳐 쓰는 것이다**.

대회 이름은 MOA @ MICRO 2026 Kernel Optimization Round이고, 잘하면 아테네에서 발표한다.

1.2 지금 어디까지 왔나

RNGD 실측 기준 (Arena job 15360, 3/3 정확도 PASS):

	sliding_project_qkv	sliding_attention_output	decoder_feedforward	기하평균
V0_baseline 실측 cycle	244,885	405,253	3,706,465	1.000×
V50 실측 cycle	149,244	56,522	354,825	—
speedup	1.64×	7.17×	10.45×	4.972×

실험 50건 중 실측으로 확정된 SOTA는 넷이다: V0 → V7(2.430×) → V14(3.936×) → V50(4.972×).

여기까지 오는 길에 한 번 크게 넘어졌다

2026-09-09 첫 실측 전까지 이 프로젝트는 **makespan**(컴파일러가 짜놓은 계획표의 길이)만 보고 달렸고, 그 위에서 5.801×까지 올려놨었다. 첫 Arena 제출에서 **V15 이후 전 계보가 정확도 FAIL**로 드러났다. 원인은 성능 모델이 아니라 **틀린 코드**였다 — 자세한 건 §9.2.

살아남은 것을 추려 다시 세운 게 V50이고, 그게 지금의 4.972×다. 이 보고서의 성능 수치는 **전부 실측으로 교체**했다.

남은 병목은 **qkv 하나**다. 1.64×는 나머지 둘(7.17× / 10.45×)에 크게 못 미친다. 세 번의 SOTA 갱신을 통틀어 qkv는 1.29× → 1.34× → 1.64×로만 움직였다 — **한 번도 제대로 공략된 적이 없다**. 기하평균이므로 다음 레버리지는 전부 거기 있다(§5.2).

2. NPU가 어떻게 동작하는가 — 30분 속성 과정

2.1 CPU, GPU, 그리고 NPU

셋 다 곱셈과 덧셈을 하는 기계인데, "한 번에 몇 개를, 얼마나 자유롭게"가 다르다.

	한 번에 처리	잘하는 것	못하는 것
CPU	몇 개 (코어 수만큼)	뭐든지. 분기, 포인터, 조건문	대량 병렬
GPU	수천 개 (스레드)	같은 연산을 데이터만 바꿔 반복	스레드마다 다른 일
NPU (RNGD)	수십만 개	텐서 연산 하나	그 외 전부

NPU는 극단으로 간 특수 목적 기계다. 대신 그 하나를 아주 잘한다. RNGD는 BF16 기준 초당 256조 번 곱셈-덧셈 (256 TFLOPS)을 한다.

여기서 중요한 차이 하나. GPU에서 CUDA를 쓰면 "스레드 100만 개를 띄워라"라고 쓰고 스케줄링은 하드웨어가 알아서 한다. **RNGD에서는 그 배치를 프로그래머가 직접 쓴다**. 어떤 데이터를 어느 메모리 층에 놓을지, 어느 연

산 유닛에 태울지, 몇 개씩 쪼갤지를 전부 코드로 지정한다. 그래서 이 대회가 성립한다 — 같은 수학인데 배치를 바꾸면 5배가 달라진다.

2.2 contraction이 뭔가 — 행렬곱의 일반화

RNGD의 유일한 원시 연산이 텐서 contraction이다. 이름이 낯설지 뭐지, 사실 이미 아는 것이다.

행렬 × 벡터를 생각해보자.

$$y_i = \sum_j W_{ij} \cdot x_j$$

여기서 벌어지는 일은 세 단계다.

- 1. W_{ij} 와 x_j 를 짝지어 곱한다 (같은 j 끼리)
- 2. j 에 대해 전부 더한다
- 3. 남은 i 가 결과의 축이 된다

"두 텐서를 공통 축에서 곱하고 그 축을 따라 더해서 없애는 것" — 이게 contraction이다. 사라지는 축(j)을 접힌 다(contracted)고 한다. 행렬곱, 행렬-벡터곱, 내적, 배치 행렬곱, 컨볼루션이 전부 이 하나의 틀에 들어간다. 어느 축을 접느냐만 다르다.

내적	$a \cdot b$	$= \sum_i a_i b_i$	(i 를 접음, 남은 축 없음 → 스칼라)
행렬×벡터	$y = Wx$	$= \sum_j W_{ij} x_j$	(j 를 접음, i 가 남음)
행렬×행렬	$C = AB$	$= \sum_k A_{ik} B_{kj}$	(k 를 접음, $i \cdot j$ 가 남음)
어텐션 점수	$S = QK^T$	$= \sum_d Q_{hd} K_{td}$	(d 를 접음, $h \cdot t$ 가 남음)

GPU와의 차이가 여기 있다. GPU는 이 모든 걸 2D 행렬곱(GEMM)으로 바꿔서 처리한다. 4차원 텐서가 들어오면 reshape/transpose로 2D로 눌러 편 다음 GEMM을 부르고 다시 펴는데, 그 눌렀다 펴는 과정 자체가 메모리를 왕복하는 비용이다. RNGD는 다차원 contraction을 그대로 실행한다. 그래서 이름이 Tensor Contraction Processor(TCP)다.

우리 커널에서 실제로 접히는 축은 이렇다.

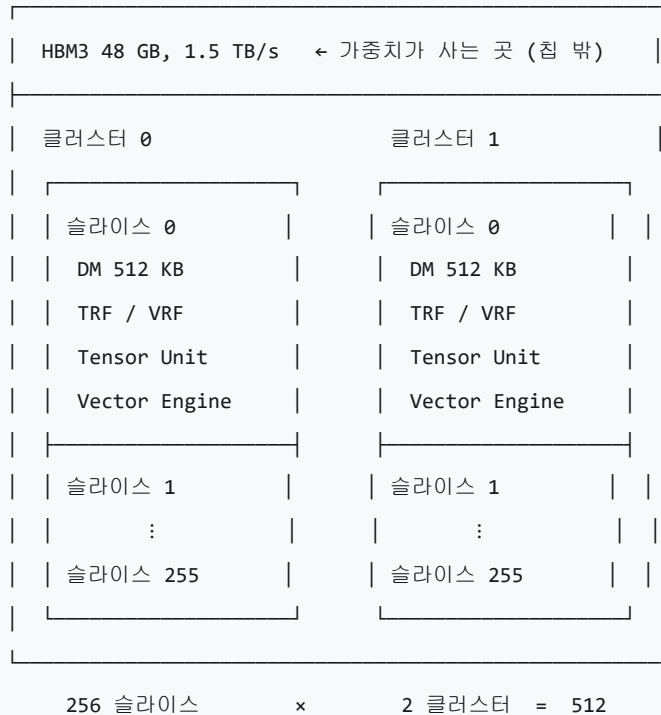
연산	식	접히는 축	크기
Q 투영	$q[o] = \sum_h Wq[o,h] \cdot x[h]$	H	3,840
K/V 투영	$k[o] = \sum_h Wk[o,h] \cdot x[h]$	H	3,840
O 투영	$y[h] = \sum_o Wo[h,o] \cdot \text{attn}[o]$	Qs	4,096
FFN up/gate	$u[l] = \sum_h Wu[l,h] \cdot x[h]$	H	3,840
FFN down	$y[h] = \sum_l Wd[h,l] \cdot g[l]$	L	15,360

전부 행렬 × 벡터다. 토큰 하나씩 처리하기 때문이다(\$2.6에서 다시 다룬다).

2.3 RNGD의 몸 — 512개의 작은 컴퓨터

칩을 안에서부터 밖으로 그리면 이렇다.

RNGD 칩 하나



슬라이스(slice)가 기본 작업 단위다. 각자 512 KB짜리 전용 메모리(DM)와 연산기를 갖는다. $512 \times 512 \text{ KB} = 256 \text{ MB}$ — 스펙시트의 온칩 SRAM 용량과 정확히 맞는다.

행렬 $\times$ 벡터를 512개로 나누는 방법은 자연스럽다. **출력 행을 나눠 갖는다.**

```
q[0..4095] = Wq[4096 × 3840] · x[3840]
```

슬라이스 0 → Wq의 0~7번 행 담당 → q[0..7] 계산

슬라이스 1 → Wq의 8~15번 행 담당 → q[8..15] 계산

:

슬라이스 511 → Wq의 4088~4095행 담당 → q[4088..4095] 계산

각 슬라이스는 **자기 행만** 갖고 있으면 되는데, **입력 x는 전부가 똑같이 필요하다**. 512개 슬라이스 전부에 3,840 개짜리 벡터를 복사해줘야 한다. 이 복사가 공짜가 아니고, 실제로 이 프로젝트에서 가장 먼저 잡은 병목이었다 (§9, V7).

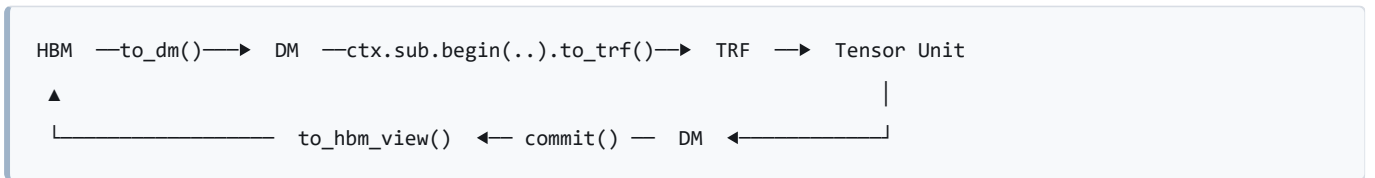
함정 하나. v0_baseline의 코드는 `cluster = m![1 # 2]`로 되어 있었는데, 이게 "논리적으로 1개를 물리적으로 2개 자리에 복제"라는 뜻이다. 즉 **두 클러스터가 똑같은 계산을 하고 있었다** — 칩의 절반이 놀고 있었다. 이걸 고치는 게 v14_two_clusters다.

2.4 메모리 5층 구조

속도와 크기가 반비례하는 계단이다. 도서관에 비유하면 이렇다.

층	크기	비유	코드 타입
HBM	48 GB	서고 — 다 있지만 가지러 가야 함	HbmTensor
DM	512 KB × 512	내 책상 — 슬라이스마다 하나씩	DmTensor
TRF	작음	펼쳐놓은 책 — contraction 피연산자	TrfTensor
VRF	작음	손에 든 메모지 — 벡터 상수	VrfTensor
레지스터	—	지금 보는 줄	(직접 안 다룸)

데이터는 항상 이 계단을 타고 오르내린다.



계단을 몇 번 오르내리느냐가 곧 성능이다. 이 프로젝트 최적화의 절반은 "왕복 한 번 줄이기"였다.

2.5 세 명의 일꾼 — 실행 컨텍스트

RNGD 안에는 동시에 일할 수 있는 일꾼이 셋 있다. 코드에서 `ctx.main`, `ctx.sub`, `ctx.tdma` 로 부른다.

일꾼	코드	하는 일
MainContext	<code>ctx.main</code>	주력. contraction, 벡터 연산, 타입 변환, 슬라이스 간 데이터 이동
SubContext	<code>ctx.sub</code>	조수. 레지스터 파일에 미리 올려놓기 (<code>to_trf</code> , <code>to_vrf</code>)
DmaEngine	<code>ctx.tdma</code>	짐꾼. HBM ↔ DM, DM ↔ DM 이동

규칙 세 개만 기억하면 된다.

규칙 1 — 같은 일꾼에게 시킨 일은 순서대로 처리된다. `ctx.main` 에 A, B, C를 시키면 A→B→C다. 아무리 A와 B가 독립이어도 겹치지 않는다.

규칙 2 — 다른 일꾼끼리는 겹친다. 짐꾼이 가중치를 나르는 동안 주력이 계산할 수 있다. 단, 서로 같은 데이터를 건드리면(의존) 기다려야 한다.

규칙 3 — MainContext와 SubContext는 같은 파이프라인을 공유한다. 완전히 겹치지는 않는다. 최악의 경우 둘의 합, 잘되면 큰 쪽 시간에 수렴한다.

여기서 **핵심 최적화 아이디어** 하나가 바로 나온다.

한 일꾼이 96% 바쁘고 나머지가 놀고 있다면, 일을 옮기기만 해도 빨라진다.

`v0_baseline` 이 정확히 그 상태였다. `MainContext` 가 96%를 차지했고, 짐꾼(DMA)은 대부분 놀고 있었다.

2.6 왜 LLM 디코딩은 "느린 게 정상"인가 — 루프라인

이게 이 프로젝트에서 제일 중요한 개념이다. 천천히 가자.

LLM은 토큰을 하나씩 만든다. "안녕"을 만들고, 그걸 다시 입력에 넣어 "하세요"를 만든다. 순차적이라 건너뛸 수 없다. 그래서 한 번의 계산에 들어가는 입력은 **토큰 딱 하나 = 3,840개짜리 벡터 하나다.**

그런데 그 벡터 하나를 처리하려고 **가중치 행렬 전체를 메모리에서 읽어와야 한다.**

```
sliding_project_qkv 한 번 실행:
  읽는 데이터: Q·K·V 가중치  30 MB
  하는 계산:    31,500,000 번의 곱셈-덧셈
```

이 둘의 비율을 **산술 강도(arithmetic intensity)**라고 한다.

$$\text{산술 강도} = \frac{\text{연산 횟수}}{\text{읽은 바이트}} = \frac{2 \times 31.5\text{M}}{30\text{MB}} \approx 2.0 \text{ FLOP/byte}$$

바이트 하나 읽어와서 곱셈 두 번 하고 버린다. 이제 하드웨어 쪽 비율을 보자.

$$\text{RNGD의 균형점} = \frac{256 \text{ TFLOPS}}{1.5 \text{ TB/s}} \approx 170 \text{ FLOP/byte}$$

170이 필요한데 2를 준다. 85배 부족하다. 결론은 명확하다.

이 커널들은 계산이 느려서 느린 게 아니라, 데이터를 못 갖다 줘서 느리다. 연산기는 대부분의 시간을 놀면서 기다린다. 이런 상태를 **메모리 바운드(memory-bound)**라고 한다.

숫자로 확인해보자. 1 GHz니까 **1.5 TB/s = 사이클당 1,500 바이트**, **256 TFLOPS = 사이클당 128,000 MAC**이다.

	데이터를 다 읽는 데	계산을 다 하는 데
sliding_project_qkv	~21,000 cycle	~250 cycle
sliding_attention_output	~10,500 cycle	~125 cycle
decoder_feedforward	~66,000 cycle	~1,400 cycle

읽는 시간이 계산 시간의 50배 안팎이다. 그러니 "행렬곱을 빠르게" 하는 최적화는 여기서 의미가 거의 없다. 이미 연산기는 놀고 있다.

그런데 실제 **baseline**은 그보다도 훨씬 느렸다. (아래는 전부 RNGD 실측 cycle이다)

	이론 하한	V0_baseline 실측	배수	V50 실측	배수
sliding_project_qkv	~21,000	244,885	11.7배	149,244	7.1배
sliding_attention_output	~10,500	405,253	38.6배	56,522	5.4배
decoder_feedforward	~66,000	3,706,465	56.2배	354,825	5.4배

`m![1 # 256]` → 값 1개를 256칸짜리 자리에 놓는다 = 256개 슬라이스에 똑같이 복제
`m![H % 8 # 16]` → 8개를 16칸 자리에 놓는다 = 나머지 8칸은 패딩

`A # B` 에서 **A**는 실제 데이터 개수, **B**는 차지하는 물리 공간이다. `A < B` 면 남는 자리는 복제되거나 비어 있다.

`Cluster = m![1 # 2]` 가 "1개 클러스터 분량의 일을 2개 클러스터 자리에 복제" = 절반이 노는 상태였던 이유다. `v14` 는 이걸 `m![Qs / 2048]` (= 2, 진짜 논리적 2개)로 바꾼다.

③ = — 부분만 잘라 본다

`m![H = 480 # 3840]` → 전체 3840 중 480칸짜리 조각

`tile()` 로 큰 텐서의 일부만 볼 때 쓴다.

3.2 커널 체인 해부 — 실제 코드 한 덩어리

`src/device/sliding/projection.rs` 의 Q 투영 핵심부를 한 줄씩 읽어보자. 이 하나만 이해하면 나머지는 같은 패턴의 변주다.

```
let contraction = ctx

    .main                                // ① 주력 일꾼에게 시킴
    .begin(weight_f8.view())             // ② 입력: DM에 있는 f8 가중치
    .fetch::
```

읽는 순서. `begin()` 으로 시작해서 `.commit()` 으로 끝나는 하나의 **파이프라인 선언**이다. 각 단계가 실제 명령어가 되고, 컴파일러가 이걸 스케줄로 배치한다.

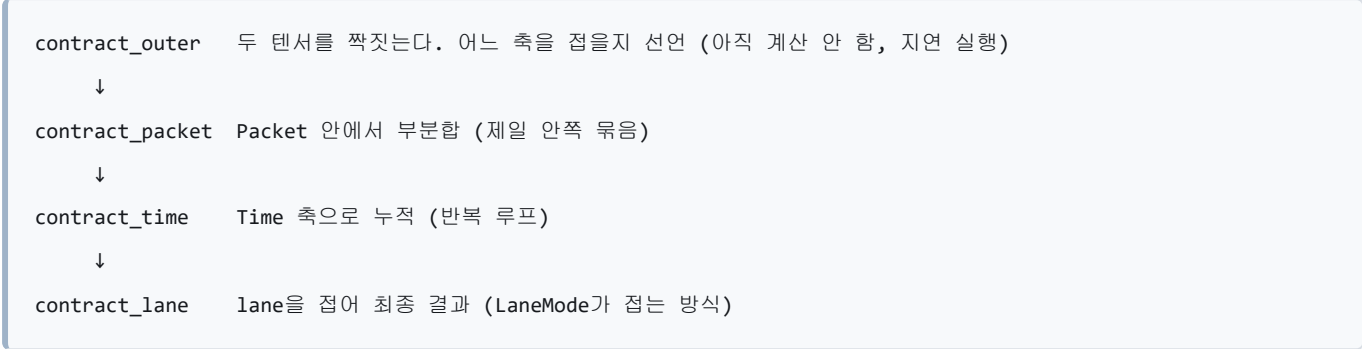
핵심 3단.

- **fetch** — DM에서 데이터를 꺼낸다. 여기서 **타입 변환을 같이 할 수 있다**(④). `f8` 가중치를 읽으면서 `bf16`으로 퍼는 건데, 이걸 따로 하면 패스가 하나 더 생긴다. 이 융합이 실제 최적화였다(\$9, V10).
- **contract_*** — §3.3에서 설명한다.
- **commit** — 결과를 DM에 쓴다. `commit_view()` 를 쓰면 큰 텐서의 특정 위치에 바로 쓸 수 있다(중간 복사 생략).

중간 결과는 전부 DM을 거친다. `commit()` 은 곧 "DM에 쓰기"이고, 다음 체인이 그걸 다시 읽는다. 그래서 체인 개수 = DM 왕복 횟수이고, 체인을 합치는 게 곧 최적화다.

3.3 4단 contraction 파이프라인

⑥~⑨가 §2.2에서 말한 "곱하고 접기"를 하드웨어 단계로 나눈 것이다.



$\sum_h w[o,h] \cdot x[h]$ 에서 h 가 3,840개인데 하드웨어가 한 번에 접을 수 있는 건 그보다 작다. 그래서 3,840개를 packet → time → lane 세 단계로 나눠서 접는다. ⑥의 $m! [H / 32]$, $m! [H \% 32]$ 가 "32개씩 묶어서 120번"이라는 뜻이다.

수학적으로는 그냥 덧셈의 결합법칙이다.

$$\sum_{h=0}^{3839} = \sum_{lane} \sum_{time} \sum_{packet}$$

어떻게 나누느냐가 자유도이고, 그게 성능을 바꾼다. 32개씩 120번이 나은지 64개씩 60번이 나은지는 하드웨어 자원 경합에 달려 있어서, 결국 재보는 수밖에 없다.

4. 최적화 원시 카탈로그

이 프로젝트에서 실제로 통한(또는 안 통한) 레버들이다. 각각 "무엇을 / 언제 / 실제 사례"로 정리했다. §9의 실험 목록이 전부 이 아홉 개의 조합이다.

원시 1. 융합 (fuse) — 두 패스를 한 체인으로

무엇을. 따로 하던 두 단계를 하나의 `begin()...commit()` 체인에 합친다.

왜 되나. 체인이 끝나면 결과가 DM에 써지고, 다음 체인이 그걸 다시 읽는다. 합치면 쓰기 한 번 + 읽기 한 번이 통째로 사라진다.

```
// 전: 두 체인 = DM에 30 MB 쓰고 다시 읽음
let widened = ctx.main.begin(weight_f8).fetch_table_lookup::<bf16>().commit();
let result = ctx.main.begin(widened).contract_outer(&x_trf)...commit();

// 후: 한 체인 = fetch 단계에서 변환하며 바로 접음
let result = ctx.main.begin(weight_f8)
    .fetch::<...>()
    .fetch_table_lookup::<bf16>() // ← 여기로 흡수
    .contract_outer(&x_trf)...commit();
```

실제 사례. `V10_attn_weight_tiles_fused_lut` — f8→bf16 룩업을 qkv/attn_out의 contraction 체인에 융합. attn_out 53,848, qkv 93,127로 내려갔다.

한계. 아무거나 합쳐지지 않는다. 하드웨어 파이프라인 단계 순서를 지켜야 한다(fetch 단계 변환은 되지만, contraction 뒤 벡터 연산 뒤 또 contraction은 안 된다).

원시 2. 재배포 (relayout) — 일을 몇 조각으로 나눌지 바꾸기

무엇을. 같은 데이터를 다른 슬라이스 분할로 놓는다.

왜 되나. 512개 슬라이스가 있는데 32개만 쓰고 있으면 16배 손해다. 반대로 너무 잘게 쪼개면 조각당 고정 비용 (스케일 로드, 커밋)이 조각 수만큼 붙는다. **중간 어딘가에 최적점이 있다.**

```
// 전: 32 슬라이스 × 120행
type HiddenRows = m![H / 120, 1 # 8];

// 후: 256 슬라이스 × 15행 + 슬라이스 간 합산
type HiddenRows = m![H / 60, ...];
```

실제 사례. - `V2_attnout_rows_over_256_slices` — O 투영을 32 → 256 슬라이스로. **194,020 → 106,461 (45% 감소)**. 이 프로젝트 최대 단일 개선. - `V12_ffn_rows_per_pass_12` — FFN 패스당 4행 → 12행. 패스 수가 1/3로 줄어 고정 비용 감소. - `V14_two_clusters` — 클러스터 1개 → 2개. 512 슬라이스 전부 사용.

한계. 슬라이스를 늘리면 각자 담당이 작아져서 **슬라이스 간 합산(inter-slice reduce)**이 필요해진다. 그 비용이 이득을 넘으면 역효과다.

원시 3. 엔진 이동 (offload) — 놓고 있는 일꾼에게 넘기기

무엇을. `ctx.main` 이 하던 일을 `ctx.tdma` 나 `ctx.sub` 로 옮긴다.

왜 되나. §2.5 규칙 1 — 같은 일꾼의 일은 직렬이다. 96% 바쁜 일꾼에게서 일을 덜어내면 그만큼 총 시간이 준다.

실제 사례. `v0` 에서 `MainContext` 점유율이 96%였고, 최장 노드가 `layout.rs` 의 브로드캐스트(약 62,000 cycle)였다. 이걸 옮기는 게 V7의 출발점이었다.

한계. 옮긴 곳이 새 병목이 될 수 있다. 옮기고 나서 지배 컨텍스트가 뭘로 바뀌었는지 반드시 다시 본다.

원시 4. 경유 (staging) — 빠른 우회로 찾기

무엇을. A에서 B로 직접 가는 대신, C를 거쳐 간다.

왜 되나. 직관에 반하는데, 온칩 데이터 이동이 HBM 왕복보다 느릴 수 있다. 슬라이스 512개에 값을 뿌리는 switch 는 링 구조로 256홉을 도는 연산이라 비싸다. 반면 HBM에서 DM으로 읽어오는 DMA는 "브로드캐스트 읽기"를 하드웨어가 원래 지원한다.

```
// 전: 온칩에서 512 슬라이스에 뿌리기 = 54,000~62,000 cycle
let x = layout::broadcast_hidden(ctx, &x);

// 후: HBM에 7.5 KB 썼다가 복제 로드 = HBM DMA 속도
let mut x_hbm: HbmTensor<bf16, Chip, m![H]> = HbmTensor::new();
x.view().to_hbm_view(&mut ctx.tdma, x_hbm.view_mut());
let x = x_hbm.to_dm(&mut ctx.tdma); // 512 슬라이스에 복제되어 로드됨
```

실제 사례 (실측으로 확인됨). - `v7_qkv_x_replicate_via_hbm` — 위 코드 그대로. 실측 기하평균 $1.000\times \rightarrow 2.430\times$, 3/3 PASS. 이 프로젝트 최초의 검증된 승리다. - `v9_ffn_x_via_hbm`, `v13_ffn_dma_trims` — 같은 수법을 FFN에 적용.

그리고 여기서 이 프로젝트 최대의 사고가 났다.

`v15_x_replicate_hbm_copies` 는 한 단계 더 나가려 했다. V7의 HBM 경유마저도 512개 슬라이스가 같은 7.5 KB 주소를 동시에 읽는 패턴이라 420 B/cycle밖에 안 나왔으니, 같은 벡터를 HBM에 8부 써두고 슬라이스마다 다른 사본을 읽게 하자는 것이었다. makespan이 실제로 줄었다(qkv 73,445 $\rightarrow$ 60,412).

그런데 사본이 한 번도 만들어지지 않았다. 코드는 "목적지 텐서에 사본 축을 붙이면 DMA write가 그 축을 따라 복제된다"고 가정했는데, 복제되지 않는다. 사본 1개만 채워지고 나머지 7부는 초기화되지 않은 HBM이었다. 슬라이스 8개 중 7개가 쓰레기를 x로 읽고 있었던 것이다.

makespan은 이걸 못 잡는다 — 정적 스케줄은 "얼마나 걸리는가"에 답하지 "맞는가"에 답하지 않는다. V15부터 V41까지 26세대가 이 위에 쌓였고, 첫 실측에서 전부 정확도 FAIL로 날아갔다(\$9.2).

교훈 셋. 1. "온칩이 무조건 빠르다"는 통념은 틀렸다 — V7이 반례다. 2. 중복 저장이 정당일 수 있다는 아이디어 자체는 살아 있다. 다만 사본을 진짜로 만들려면 사본마다 `to_hbm_view` 를 명시적으로 불러야 하고, 그러면 store 하나당 Core 디스크립터 명령이 3개씩 붙는다. Core는 in-order 발행이라 그게 임계경로가 된다 — 사본 수 1/2/4/8/16을 전수 측정한 결과 어디에서도 사본 없는 쪽을 못 이겼다(V49, 기각). 3. 성능이 좋아졌다는 신호를 정확도 확인 없이 믿으면 안 된다. 특히 "공짜로 늘어난 것"처럼 보일 때.

원시 5. 청킹 (chunking) — 필요한 만큼만 풀기

무엇을. 전체를 한꺼번에 처리하는 대신 조각내서, 각 조각이 필요할 때만 준비한다.

왜 되나. 압축된 가중치를 전부 풀어놓으면 DM이 넘치고, 넘치면 DM↔DM 재배포 DMA가 생긴다. 조각내면 그 재배포가 통째로 사라진다.

실제 사례. `V1_ffn_down_chunked_dequant` — down 투영에서 슬라이스별로 `L/8` 청크만 역양자화. **1,693,200 → 609,223 (64% 감소)**. FFN 최대 개선.

한계. 조각마다 고정 비용이 붙는다. 원시 2와 정확히 같은 트레이드오프다.

원시 6. 오버랩 (overlap) — 독립적인 일을 겹치기

무엇을. 서로 의존하지 않는 두 작업을 다른 일꾼에게 나눠 동시에 돌린다.

왜 되나. 점수가 되는 cycle은 **모든 작업 구간의 합집합**이다(§8.2). 겹치면 그만큼 줄어든다.

```
전: [up 가중치 로드][up 계산][gate 가중치 로드][gate 계산]
후: [up 가중치 로드][up 계산]
    [gate 가중치 로드][gate 계산]    ← DMA가 앞당겨짐
```

실제 사례. `V6_ffn_upgate_overlap` — up과 gate는 완전히 독립인데 순차 실행되고 있었다. 609,223 → 412,304.

한계. MainContext끼리는 못 겹친다(규칙 1). 한쪽을 DMA/Sub로 밀어낼 수 있을 때만 성립한다.

원시 7. 폭 확장 (widen) — 놓고 있는 하드웨어 쓰기

무엇을. 안 쓰고 있던 자원을 동원한다.

실제 사례. `V14_two_clusters` — `Cluster = m![1 # 2]` 라 두 클러스터가 같은 계산을 반복하고 있었다. 행을 실제로 나눠 512 슬라이스를 쓰자 **세 커널이 동시에 1.27× / 1.31× / 1.83×** 빨라졌고, 기하평균이 2.87× → 4.15×로 뛰었다. **이 프로젝트 단일 최대 개선이다.**

한계. 클러스터 간 데이터 합치기가 새로 필요해진다. 현재 코드는 그걸 HBM 경유(원시 4)로 푼다 — 512 슬라이스에서 16 B씩 직접 저장하면 37,000 cycle이 든다고 코드 주석에 적혀 있다.

교훈. 미세 튜닝을 열 번 하기 전에 **"자원을 다 쓰고 있는가"부터 확인해야 한다.** V1~V13이 13번의 실험으로 2.87×를 만드는 동안, 놓고 있던 절반을 켜는 한 번이 1.45×를 더 얻었다.

원시 8. 정밀도 예산 (precision budget) — 오차 여유를 성능으로 바꾸기

무엇을. 중간 계산의 f32 왕복을 bf16으로 줄인다.

왜 되나. `bf16 → f32 → 연산 → bf16` 왕복은 변환 비용 + 메모리 2배다. 정확도 여유가 있으면 생략할 수 있다.

커널마다 여유가 다르다.

커널	atol	여유
sliding_attention_output	0.05	가장 넓음
sliding_project_qkv	0.04	넓음
decoder_feedforward	0.01	거의 없음

실제 사례. 처음엔 "맨 마지막에 쓸 레버"로 미뤄졌는데, 결과적으로 후반부(V29~V35)의 주력이 됐다. 다만 쓰인 방식이 예상과 달랐다 — **정밀도를 낮춰서 얻은 게 아니라, 수학적으로 동일한 재배치를 하기 위한 여유로 썼다.**

- **v29** — FFN 블록 스케일을 가중치가 아니라 **부분합에 적용**. $\sum_b s_b \sum_{j \in b} w_j x_j = \sum_j (w_j s_b) x_j$ 로 수학적으로 같고, 달라지는 건 f32 합산 순서뿐이다. 벡터 엔진 부하 121k → 45k 예상.
- **v33** — 어텐션 출력을 상수 16으로 스케일. 어텐션 출력은 $\sum_j p_j v_j$ 이고 $\sum p_j = 1, p_j \geq 0$ 이라 값의 범위가 미리 묶여 있다. **동적으로 계산하던 스케일을 상수로 바꿔서** 계산 체인 전체를 지웠다.

주의. 정확도는 hard gate다. 틀리면 아무리 빨라도 **0점**이다(\$8.3). 위 변경들은 전부 **makespan**에서만 검증됐고 **실측 정확도는 미확인**이다. RESULTS.md에도 "실측 검증 필요"로 적혀 있다.

원시 10. 분해 (split) — 느린 타입을 빠른 타입 둘로 쪼개기

무엇을. bf16 값 하나를 f8 두 조각(hi, lo)으로 나눠서, 느린 bf16 경로 대신 빠른 f8 경로를 두 번 탄다.

왜 되나. §2.1에서 봤듯 RNGD는 FP8이 BF16의 2배다. 그런데 $f8 \times f8$ contraction을 하려면 **양쪽 다 f8**이어야 하는데, 활성값은 bf16이다. 그래서 활성값을 $x \approx (hi + lo) \cdot 2^{-k}$ 로 쪼갠 뒤 가중치 패킷을 시간축으로 두 번 흘린다. 정밀도는 두 조각의 합으로 복원된다.

$$\sum_j w_j x_j = 2^{-k} \left(\sum_j w_j hi_j + \sum_j w_j lo_j \right)$$

결정적인 부수 효과. f8로 직접 곱하면 **f8→bf16 LUT 패스가 통째로 사라진다**. 원시 1이 LUT를 contraction에 융합했다면, 이건 LUT 자체를 없앤다.

실제 사례. **v31** (qkv) — LUT 패스 5,063과 그에 딸린 DMA가 사라지고 K/V contraction이 $2,663 \rightarrow 1,225$. **v32** (attn_out) — 같은 수법, $30,037 \rightarrow 29,318$. **v29** (FFN)에서 먼저 쓰인 기법이다.

공짜로 얻은 것. 결과가 2^k 배로 커진 채 나오는데, **뒤따르는 RMSNorm이 스케일 불변**이라($\text{rms}(s \cdot q) = s \cdot \text{rms}(q)$) 되돌릴 필요가 없다. eps 항만 s^2 분의 1로 작아지는데 상대 오차 $1e-6$ 이하다. 정규화가 뒤에 있는 구조라서 성립하는 트릭이다.

한계. hi/lo 분해는 $|x| > \max |x|/4096$ 구간에서만 exact고, 그 아래는 $2^{-10}/s$ 수준의 절대 오차가 난다. 그리고 hi/lo를 HBM에 두 번 저장해야 해서 store가 늘어난다 — 그걸 한 번으로 줄이려던 **v39**는 컴파일 실패했다(\$9.4).

⚠ 이 원시는 아직 실측 검증이 안 됐다. V29·V31·V32는 전부 V15 계보 위에 얹혀 있어서 첫 실측에서 사본 버그와 함께 FAIL 처리됐다. 분해 기법 **자체**가 틀렸다는 증거는 없다 — 오염원은 사본이었고, 분해는 그 위에 서 있었을 뿐이다. V50 위에 다시 얹어 실측해봐야 안다(\$9.3).

원시 9. 중복 제거 (dedup) — 같은 일 두 번 하지 않기

무엇을. 이미 되어 있는 걸 다시 하는 코드를 찾아 지운다.

실제 사례(부분 성공). `shared::rmsnorm::normalize` 의 마지막 단계가 이미 256 슬라이스에 복제해서 돌려주는데, `ops.rs` 가 곧바로 `broadcast_hidden` 으로 또 복제하고 있었다. 다만 타입만 재라벨링하는 것으로는 안 됐고(물리 배치가 기대와 달랐다), 결국 원시 4(HBM 경유)로 우회해서 풀었다.

교훈. 코드를 보고 "중복 같다"고 판단해도 하드웨어 배치는 다를 수 있다. 스케줄 dump로 확인하는 게 먼저다.

원시 요약표

검증 상태를 같이 표시한다.  = 실측 PASS 계보에 포함,  = makespan만 (V15 계보에 얹혀 있어 재검증 필요),  = 실측으로 기각.

#	원시	한 줄	실제 사례	검증	효과
1	융합	두 체인을 하나로	V10 / V16, V18, V23, V36, V38		대
2	재배치	슬라이스 분할 바꾸기	V2 / V12, V19, V20, V22, V28, V37	 	대
3	엔진 이동	놓고 있는 일꾼에게	(V7의 동기)		중
4	경유	HBM 우회가 더 빠름	V7, V9, V13 / V15, V24	 	대
5	청킹	필요한 만큼만 풀기	V1		대
6	오버랩	독립 작업 겹치기	V6, V17		중
7	폭 확장	놀던 하드웨어 쓰기	V14, V25		최대
8	정밀도 예산	오차 여유로 재배치 허가받기	V29, V33, V35		대
9	중복 제거	두 번 하지 않기	(V7로 흡수)	—	—
10	분해	bf16을 f8 둘로 쪼개 빠른 경로	V29, V31, V32		대
11	되돌리기	틀린 전제에 기댄 것을 빼기	V50		대

패턴이 보인다. 큰 효과를 낸 것은 전부 "데이터를 어떤 타입으로, 어디에, 몇 조각으로 놓고, 몇 번 옮길지"를 바꾼 것이다. 곱셈 횟수는 처음부터 끝까지 한 번도 줄이지 않았는데 실측 4.97배가 나왔다. §2.6의 결론 그대로 — 메모리 바운드 워크로드에서는 배치가 전부다.

실측으로 확정된 세 번의 SOTA 갱신만 떼어보면 이렇다.

갱신	주력 원시	기하평균
V7 (V1+V2+V7)	5 청킹 · 2 재배치 · 4 경유	1.000 → 2.430×
V14	7 폭 확장	2.430 → 3.936× (+62%)
V50	11 되돌리기	3.936 → 4.972× (+26%)

가장 큰 두 번이 튜닝이 아니었다. 하나는 놓고 있던 클러스터를 켜 것이고, 다른 하나는 틀린 코드를 지운 것이다. 미세 튜닝 열세 번(V1~V13, 2.43×)보다 그 둘이 컸다.

원시 11(되돌리기)이 카탈로그에 있는 게 이상해 보이지만, 최적화 프로젝트에서 실제로 가장 자주 필요한 동작이다. 쌓아 올린 것 중 어느 층이 썩었는지 찾아 빼는 일 말이다.

5. 대회 규칙과 점수

5.1 일정

항목	날짜
등록 마감	2026-09-15
Stage 1 (커널)	2026-09-01 ~ 09-25
결선 발표	09-30
Stage 2 (E2E)	10-01 ~ 10-25
시상 (MICRO 2026, Athens)	11-01

5.2 점수 = 세 커널 speedup의 기하평균

$$\text{점수} = \sqrt[3]{\frac{c_{qkv}^{base}}{c_{qkv}} \times \frac{c_{attn}^{base}}{c_{attn}} \times \frac{c_{ffn}^{base}}{c_{ffn}}}$$

기하평균이라는 게 전략을 규정한다.

시나리오	qkv	attn_out	ffn	점수
하나만 2배	2.00	1.00	1.00	1.260
셋 다 1.3배	1.30	1.30	1.30	1.300
하나 2배 + 하나 20% 퇴보	2.00	1.00	0.80	1.170

"한 커널 2배"보다 "세 커널 30%씩"이 낫다. 그리고 어디 하나를 퇴보시키면 다른 곳의 성과를 통째로 까먹는다.

지금 우리 상황에 대입해보면 이렇다.

	현재 (V50 실측)	qkv만 2배 되면	ffn만 2배 되면
qkv	1.64×	3.28×	1.64×
attn_out	7.17×	7.17×	7.17×
ffn	10.45×	10.45×	20.89×
기하평균	4.97×	6.26×	6.26×

어느 커널을 2배 만들든 기하평균에 미치는 효과는 똑같다. 기하평균이 곱의 세제곱근이라 그렇다. 그러니 "어디가 제일 느린가"가 아니라 "어디가 제일 쉽게 2배가 되는가"로 골라야 한다.

그 판단에는 §7.4의 하한 대비 배수가 쓸모 있다. V50 실측에서 qkv 7.1배 / attn_out 5.4배 / ffn 5.4배다. qkv가 셋 중 여유가 가장 크다 — 절대 speedup은 제일 낮는데 남은 군더더기는 제일 많다. 그래서 1순위다(§9.4).

이 동조 현상 자체가 정보다. 커널마다 다른 최적화를 했는데도 결과가 같이 움직였다면, 남은 여유는 개별 커널의 코드가 아니라 세 커널이 공유하는 무언가(하드웨어 배치, 톨체인 제약, 혹은 하한 추정 자체)에 걸려 있을 가능성이 크다. §10 열린 질문 5번이 이걸 묻는다.

5.3 고쳐도 되는 곳 / 안 되는 곳

★ 채점 반영 / ✕ 무시(baseline으로 되돌림)

범위	
src/device/ 전체	★
src/ops.rs , ops_vision.rs , ops_audio.rs 의 함수 본문만	★
src/axes.rs , src/host/ , src/api/ , src/bin/ , src/lib.rs , tests/	✕

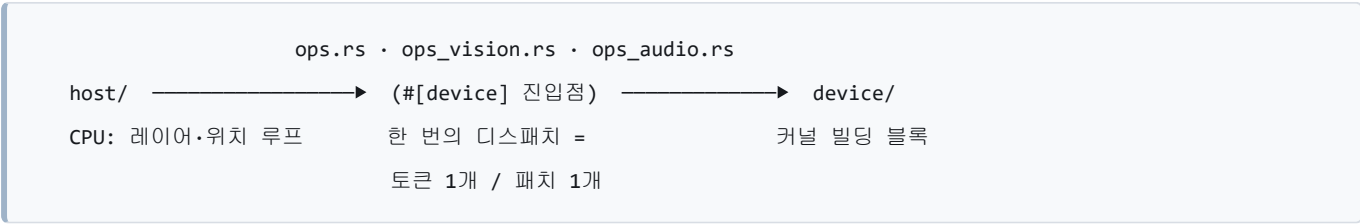
절대 규칙 넷.

- 1. #[device] 함수의 이름·인자·타입·반환형을 바꾸지 않는다. 시그니처가 평가자 계약이다.
- 2. ops*.rs 는 크레이트 루트에 그대로 둔다. 커널 이름에 module_path!() 가 들어간다.
- 3. tests/test_kernels.rs 를 고쳐서 통과시키지 않는다. 채점 서버는 자기 버전을 쓴다.
- 4. 정확도는 hard gate다. 틀리면 0점.

✕ 영역에서 빨라진 건 전부 가짜다. 설계 단계에서 배제한다.

6. 코드 지도

6.1 device / host 분리



- 커널은 작업 단위 하나만 처리한다. 배치 차원도 시퀀스 차원도 없다. 48개 레이어 루프, 토큰 위치 루프는 전부 호스트에 있다.
- ops*.rs 가 유일한 접점이다. 호스트가 부르는 #[device] 함수들.

6.2 파일별

경로	채점	내용
src/ops.rs	★	텍스트 커널 10개. Stage 1 대상 3개가 여기
src/device/layout.rs	★	Cluster / Slice / Replicated / BothClusters , 브로드캐스트 헬퍼
src/device/sliding/projection.rs	★	Q/K/V/O 투영. f8 가중치 + 채널당 bf16 스케일
src/device/sliding/rmsnorm.rs	★	헤드별 Q/K/V RMSNorm (V는 학습 가중치 없음)
src/device/sliding/rope.rs	★	$\theta=10,000$ 회전 위치 인코딩
src/device/sliding/attention.rs	★	링 KV 캐시 softmax. Stage 1 대상 아님
src/device/shared/rmsnorm.rs	★	히든 상태 RMSNorm. 세 커널 모두 사용
src/device/shared/mlp.rs	★	GeGLU FFN. NVFP4 (4비트 + 16개당 스케일 + 전역 스케일)
src/device/shared/residual.rs	★	residual add, 레이어 게이트
src/device/full/*	★	full-attention 8개 레이어. v_proj 없음, $\theta=1,000,000$
src/host/**, src/api/**, src/bin/**	×	CPU 오케스트레이션, HTTP 서버, 진입점
tests/test_kernels.rs	×	채점 기준 테스트

6.3 축(axes) 상수

축	값	의미
H	3,840	히든 크기
L	15,360	MLP 중간 크기 (= 4H)
W	262,144	어휘 크기
Ns , Gs , Ds	8, 2, 256	sliding: KV 헤드 8, 헤드당 쿼리 2, head dim 256
Qs , Ps , Ts	4,096, 2,048, 1,024	Q 폭, KV 폭, 링 캐시 길이
Gf , Df , Tf	16, 512, 512	full: 헤드 16, head dim 512, 페이지 길이

레이어 48개 = sliding 40 + full 8. **Stage 1 커널은 전부 sliding 쪽이다.**

7. 세 커널 해부

7.1 `ops::sliding_project_qkv` — 입력을 Q·K·V로 나누기

하는 일. 토큰 벡터 하나(3,840)를 정규화한 뒤 세 개의 행렬과 곱해서 Query(4,096) / Key(2,048) / Value(2,048)를 만들고, 헤드별 정규화와 위치 인코딩을 거쳐 KV 캐시에 쓴다.

```
x (3840)
├ RMSNorm                                정규화
├ 512 슬라이스에 복제                    ◀ V7: HBM 경유로 해결
├ Q 투영  Wq[4096×3840] · x              ◀ V10: LUT 융합 → V31: f8×f8로 LUT 제거
├ K 투영  Wk[2048×3840] · x
├ V 투영  Wv[2048×3840] · x
├ 헤드별 RMSNorm (Q·K는 가중치 있음, V는 없음) ◀ V36: 채널 스케일을 여기 흡수
├ RoPE (θ=10,000)                        ◀ V30: 테이블 직접 gather
└ q → 출력 / k,v → 캐시에 scatter
```

7.2 `ops::sliding_attention_output` — 어텐션 결과를 되돌리기

하는 일. 어텐션 출력(4,096)을 다시 히든 크기(3,840)로 투영하고, 정규화한 뒤 residual에 더한다.

```
attn (4096)
├ 0 투영  Wo[3840×4096] · attn            ◀ V2: 32→256 슬라이스 / V32: f8×f8
├ RMSNorm                                ◀ V18: 스케일을 에필로그로 / V33: 상수 16
├ residual + x                          ◀ V11: 480×8 → 1920×2 / V16: 융합
└ residual에 쓰기
```

세 커널 중 가장 작다(가중치 15 MB). tolerance도 가장 험겁다(0.05).

7.3 `ops::decoder_feedforward` — 가장 무거운 놈

하는 일. GeGLU 방식의 2층 MLP다. 3,840 → 15,360으로 두 갈래(up, gate) 넓히고, 게이트에 GELU를 먹여 곱한 뒤, 다시 3,840으로 좁힌다.

$$\text{FFN}(x) = W_{\text{down}}(\text{GELU}(W_{\text{gate}} x) \odot W_{\text{up}} x)$$

residual (3840)

└ RMSNorm

└ 512 슬라이스에 복제

◀ V9: HBM 경유

└ up 투영 $W_u[15360 \times 3840] \cdot x$

└ gate 투영 $W_g[15360 \times 3840] \cdot x$

◀ V6: 돌을 겹침 / V12-V37: 패스 크기

└ GeGLU: $\text{GELU}(\text{gate}) \times \text{up}$

◀ V13: HBM 경유 / V25: 두 클러스터

└ down 투영 $W_d[3840 \times 15360] \cdot g$

◀ V1: 청크 역양자화 / V29: 스케일을 부분합으로

└ RMSNorm → residual add → 레이어 게이트

◀ V16: 셋을 한 벡터 패스로 융합

└ residual에 쓰기

◀ V38: reducing 레이어아웃에서 바로 저장

가중치가 NVFP4로 저장돼 있다. 4비트로 압축하고, 16개마다 f8 스케일 하나, 행렬마다 f32 전역 스케일 하나를 곱해서 원래 값을 복원한다.

$$w_{\text{실제}} = w_{4\text{bit}} \times s_{\text{블록}} \times s_{\text{전역}}$$

이 복원(역양자화)을 매번 커널 안에서 한다. 1억 7,700만 개 원소가 $f4 \rightarrow f8 \rightarrow f32 \rightarrow x\text{스케일} \rightarrow bf16$ 을 거친다. 이게 FFN이 무거운 진짜 이유고, V1의 청킹이 통한 이유다.

7.4 비용 구조

	qkv	attn_out	ffn
가중치 (압축 상태)	30.0 MB (f8)	15.0 MB (f8)	84.4 MB (f4) + 10.5 MB (스케일)
MAC 수	31.5 M	15.7 M	176.9 M
산술 강도	2.0 FLOP/B	2.0 FLOP/B	3.56 FLOP/B
HBM 하한 (추정)	~21,000	~10,500	~66,000
V0 실측	244,885	405,253	3,706,465
V14 실측	183,005	78,708	418,837
V50 실측	149,244	56,522	354,825
하한 대비 배수	11.7 → 8.7 → 7.1	38.6 → 7.5 → 5.4	56.2 → 6.3 → 5.4
tolerance (atol)	0.04	0.05	0.01

하한은 칩 전체 대역폭(1,500 B/cycle)을 한 커널이 독점한다고 가정한 낙관치다. 절대값이 아니라 배수의 변화를 보라.

읽어야 할 것 하나 — 출발점이 제각각이었다. 베이스라인에서 qkv는 하한의 11.7배, ffn은 56.2배였다. 같은 "메모리 바운드 커널"인데 낭비의 양이 5배 달랐다는 뜻이고, 그래서 ffn이 10.45배까지 줄어드는 동안 qkv는 1.64배에 그쳤다. qkv는 원래부터 군더더기가 적었다.

읽어야 할 것 둘 — 이제 셋이 5.4~7.1배로 모였다. V14 시점(8.7 / 7.5 / 6.3)에서도 그랬고 지금도 그렇다. 이 동조는 두 가지 중 하나를 뜻한다.

- 아직 못 찾은 공통 구조가 있거나,
- 하한 추정 자체가 낙관적이거나.

두 번째가 유력하다. 우리 하한은 "바이트 ÷ 대역폭"인데, 실제 DMA 비용은 바이트가 아니라 디스크립터 개수에 묶인다는 관측이 반복해서 나왔다 — V15의 "같은 주소를 512개 디스크립터가 읽어 420 B/cycle", V39 노트의 "548 고정 + 개당 ~35", V49의 "store 하나당 Core 명령 3개". 그렇다면 남은 레버는 대역폭이 아니라 전송 횟수다.

읽어야 할 것 셋 — makespan은 성능 예측기로는 나쁘지 않았다. V50에서 makespan과 실측을 나란히 놓으면 이렇다.

	makespan이 예측한 speedup	실측 speedup
qkv	1.98×	1.64×
attn_out	7.07×	7.17×
ffn	10.70×	10.45×
기하평균	5.31×	4.972×

attn_out과 ffn은 거의 정확하고, qkv만 20% 낙관적이다. 정적 스케줄이 실패한 지점은 성능이 아니라 정확도였다 (§9.2). 이 구분이 중요하다 — makespan을 버릴 이유는 없고, 다만 makespan으로 "맞다"를 판정할 수 없을 뿐이다.

8. 측정 체계

8.1 두 가지 숫자

지표	얻는 법	성격
makespan	<code>cargo furiosa-opt compile <kernel> --exact --dump-schedule out.json → max(lifetime.end)</code>	컴파일러의 계획표 길이. 싸고 반복 가능. 점수 아님
RNGD cycles	<code>./scripts/rngd_test.sh</code> (Arena 제출)	실측. 이것만 점수다

makespan은 스크리닝용, 실측은 판정용이다. 현재 V1~V14는 전부 makespan만 있다. Arena 접근이 열리면 전부 다시 재야 한다.

8.2 실측 cycle의 정확한 정의

tests/test_kernels.rs 가 `span::npu` 스패의 `begin_cycle / end_cycle` 을 모아서,

```
fn window_cycles(&self) -> Option<u64> {
    let begin = spans.iter().map(|s| s.begin).min()?;
    let end   = spans.iter().map(|s| s.end).max()?;
    Some(end.saturating_sub(begin))
}
```

모든 스패의 합집합 구간을 그 커널의 cycle로 삼는다. 따라오는 결론 셋.

1. **디스패치 1회 전체가 측정된다.** 가중치 DMA도 포함이다. 실제 서빙이면 레이어 간 프리페치로 감출 비용도 여기선 전액 계상된다.
2. **오버랩이 그대로 점수가 된다.** 합집합이므로 겹치면 준다 → 원시 6이 유효한 이유.
3. `TUC_PROFILE_LEVEL=info` 이상일 때만 수집된다. 지연 read-back이라 기본 500ms 대기 후 읽는다.

8.3 정확도 게이트

판정식은 $|기대 - 실제| \leq atol + rtol \times |기대|$, `rtol` 은 셋 다 `1e-2`.

하나라도 넘으면 그 커널은 **0점**이고, 틀린 커널의 cycle은 기록조차 하지 않는다(`FAIL(accuracy)` 로만 남긴다).

입력은 픽스처에 저장하지 않는다. `scripts/fixture_prng.py` 와 테스트의 `prng` 모듈이 **바이트 단위로 동일한** 카운터 해시로 양쪽에서 각각 합성한다. 픽스처엔 기대 출력만 있어서 약 120 KB다.

8.4 명령어

```
# 최초 1회
python3 scripts/generate_references.py          # → ref/fixtures.safetensors

# 스크리닝 (--exact 필수: 이름이 다른 커널의 prefix일 수 있음)
mkdir -p target/schedules
cargo furiosa-opt compile ops::sliding_project_qkv --exact \
    --dump-schedule target/schedules/V{n}_sliding_project_qkv.json
cargo furiosa-opt compile ops::sliding_attention_output --exact \
    --dump-schedule target/schedules/V{n}_sliding_attention_output.json
cargo furiosa-opt compile ops::decoder_feedforward --exact \
    --dump-schedule target/schedules/V{n}_decoder_feedforward.json

# 눈으로 보기
furiosa-schedule-viewer                        # 127.0.0.1:9254

# 판정
./scripts/rngd_test.sh                        # $RNGD_URL 필요, 사전 rngd login
```

스케줄 JSON은 스크립트로 파도 된다. `instructions[]` 에 `tpe`, `contexts`, `lifetime{begin,end}`, `util{total_util}`, 소스 위치가 담긴 `description` 이 있다.

9. 지금까지의 여정과 다음

9.1 50번의 실험, 네 번의 확정

실측으로 정확도 PASS를 받은 구성만이 점수다. 그 기준에서 이 프로젝트는 네 개의 점을 지나왔다.

시점	주제	qkv	attn_out	ffn	기하평균
V0_baseline	기준선	244,885	405,253	3,706,465	1.000×
V7 (V1+V2+V7)	명백한 낭비 제거	190,342	111,005	1,213,877	2.430×
V14	놀던 클러스터 켜기	183,005	78,708	418,837	3.936×
V50	틀린 전제 되돌리기	149,244	56,522	354,825	4.972×

커널별 speedup으로 보면 서사가 한눈에 들어온다.

시점	qkv	attn_out	ffn
V7	1.29×	3.65×	3.05×
V14	1.34×	5.15×	8.85×
V50	1.64×	7.17×	10.45×

attn_out과 ffn은 7~10배가 됐는데 qkv만 제자리다.

9.2 첫 실측이 26세대를 지웠다

이게 이 프로젝트에서 가장 중요한 사건이라 따로 쓴다.

무슨 일이 있었나. Arena 접속 전까지 판정 기준은 makespan이었고, 그 위에서 V41까지 **5.801×**를 찍어놨었다. 첫 제출(job 15346~15352)에서 **V15부터 V41까지 전 계보가 정확도 FAIL로** 드러났다.

원인. V15가 도입한 HBM 사본 관용구가 “목적지 텐서에 사본 축을 붙이면 DMA write가 그 축을 따라 복제된 다”를 가정했는데 — **복제되지 않는다**. 사본 1개만 채워지고 나머지는 초기화되지 않은 HBM이었다. bf16 경로는 NaN, f8 경로는 220%대 오차가 났다.

두 번째 버그를 찾은 방법 (재사용할 가치가 있다). 사본을 걷어내고 다시 제출하니 v-attn_out·ffn은 PASS인데 q·k 만 FAIL이었고, non-finite 위치가 index 0에서 **q=641, k=385로 옮겨갔다**. `Ds=256` 이므로

$$641 = 2 \times 256 + 129, \quad 385 = 1 \times 256 + 129$$

둘 다 같은 채널 **d=129**, 같은 kv head **n=1**이다. v만 통과한다는 사실과 합치면 남는 경로는 **RoPE뿐**이고(v는 RoPE를 안 거친다), RoPE는 Ds를 128씩 반으로 나누므로 d=129는 두 번째 half의 두 번째 원소다. 여기서 공장 V30이 나왔다 — V30도 같은 전제로 cos/sin 테이블을 head 레이아웃에 직접 gather하고 있었다.

실패 위치의 인덱스는 커널 디버깅에서 가장 값싼 단서다. 스케줄을 뜨거나 중간값을 덤프하기 전에 인덱스 산수 부터 해볼 것.

두 버그가 같은 오해 하나에서 나왔다. 서로 다른 시점에 쓴 코드인데 전제가 같았다. 정적 스케줄에서 "공짜로 늘어난 것"은 대개 늘어나지 않은 것이다.

그래서 V50은 빼서 이겼다. 사본을 진짜로 채우는 길(V49)을 먼저 시도했지만, store 하나당 Core 디스크립터 명령이 3개 붙고 Core는 in-order 발행이라 사본 수 1/2/4/8/16 어디에서도 사본 없는 쪽을 못 이겼다. 아이디어 자체가 틀렸으므로 결론은 "없앤다"였다.

9.3 makespan은 무엇에 쓸 수 있나

이 사고에서 "makespan은 쓸모없다"를 끌어내면 잘못된 학습이다. §7.4에서 봤듯 V50의 makespan 예측(5.31×)은 실측(4.972×)과 꽤 가깝다 — attn_out.ffn은 1~2% 이내다.

정확한 결론은 이렇다.

makespan이 답할 수 있는 것	답할 수 없는 것
이 코드가 얼마나 걸리는가 (대략)	이 코드가 맞는가
두 배치 중 어느 쪽이 빠른가 (동일 조건)	하드웨어가 이 관용구를 실제로 지원하는가
어느 컨텍스트가 병목인가	실행 편차 (qkv는 6~7% 흔들린다)

운용 규칙. makespan은 탐색용으로 계속 쓰되, "채택" 판정은 실측으로만 한다. 그리고 새 관용구를 도입할 때는 성능이 좋아지기 전에 정확도부터 확인한다 — 26세대를 쌓기 전에 한 번만 제출했으면 됐을 일이었다.

9.4 남은 축

1순위 — qkv. 유일한 병목이다(1.64× vs 7.17× / 10.45×). 기하평균이므로 어느 커널을 2배 만들든 효과는 같은데, qkv는 하한 대비 7.1배로 셋 중 여유가 가장 크다.

주의: qkv는 실행 편차가 6~7%다(job 15359 → 159,405, job 15360 → 149,244). attn_out.ffn은 1% 이내. 하필 유일한 병목이 가장 시끄러운 커널이라, 작은 개선은 반복 측정 없이 판정할 수 없다.

2순위 — 유효한 최적화를 V50 위에 다시 쌓기. V16~V37의 대부분(융합, 재배치, 분해)은 사본 관용구와 무관하고 정확도 위험도 없다. makespan으로는 이미 5.8×까지 갔던 길이다. 다만 이번에는 한 단계씩 실측한다.

그 밖의 후보.

축	원시	내용	리스크
DMA 디스크립터 수	4	비용이 바이트가 아니라 개수에 묶인다는 관측이 반복됐다. 그렇다면 남은 레버는 전송 횟수다	중간
올바른 사본의 실측 A/B	4	V49는 makespan 기준으로 $c=1$ 을 골랐지만, 실측에서는 사본 있는 쪽이 qkv를 더 줄였다 (V38 109,309 vs V50 149,244 — 무효한 숫자지만 신호다). 실물은 $c=2$ 를 고를 수 있다	중간
V30을 정직하게 다시	9	cos/sin을 두 클러스터로 보내되 복제를 명시적으로 기록하면 된다	낮음
INT4 경로	10	FP8이 BF16의 2배라 f8 분해가 통했다. INT4는 4배고, FFN 가중치는 이미 4비트로 저장돼 있다	높음

9.5 죽은 길 (다시 가지 말 것)

실험	왜 안 됐나
V8_weight_rows_interleaved_dma	가중치 행을 교차 배치했는데 DMA 노드가 전혀 안 움직였다 . 스케줄러의 DMA 비용 모델은 슬라이스 배치 순서를 안 본다
V21_qkv_rope_tables_early	rope 테이블 gather를 앞당겼는데 불변 . 스케줄러가 이미 hoist하고 있었다
V34_attnout_scale_in_rmsnorm	스케일을 rmsnorm으로 옮겼는데 완전히 중립
V39_x2_single_store	StreamUnmatchedSegment 컴파일 실패. commit_view 는 스트림 축과 tile 축이 구조적으로 같아야 통과한다
V42_attnout_tile_shapes	20/20/20 → 28/28/4로 바꿨더니 +1,042 . 스케줄러가 4행 타일을 두 번째로 로드해서 마지막 로드가 28행이 됐다
V15 ~ V41 의 HBM 사본	없는 축은 DMA write를 복제하지 않는다 . 26세대가 여기 오염됐다
V49_hbm_copies_real_stores	사본을 제대로 채우면 store당 Core 명령 3개 × in-order 발행이 임계경로가 된다. $c=1$ 을 못 이긴다

구조적으로 배제된 것. × 영역(host/ , api/ , axes.rs , tests/) 최적화, ops::sliding_attention (Stage 1 대상 아님), #[device] 시그니처 변경, ops*.rs 이동.

틀체인 제약 (설계 단계에서 피할 것).

- **fetch LUT는 연달아 못 건다**. CanApplyFetchTableLookup 이 첫 fetch 위치에서만 성립하고 FetchCast<bf16> for f8 도 없다. NVFP4를 bf16으로 만들려면 DM 사본을 반드시 한 번 거쳐야 한다 — V29가 f8×f8 경로를 택한 이유다.
- **commit_view 는 스트림 축과 tile 축이 구조적으로 같아야 한다** (V39 실패).

10. 남은 열린 질문

1. **qkv를 어떻게 줄일 것인가?** 유일한 병목이고, 하한 대비 7.1배로 여유도 가장 크다. 그런데 실행 편차가 6~7%라 측정 자체가 까다롭다.
2. **V16~V37을 V50 위에 다시 쌓으면 실측으로 얼마가 나오는가?** makespan으로는 5.8×까지 갔던 길인데, 그중 얼마가 진짜인지 모른다.
3. **하한 대비 5.4~7.1배가 아직 남아 있다. 이건 뭔가?** "DMA 비용은 디스크립터 개수" 가설이 맞다면 하한 추정치를 고쳐야 하고, 그러면 남은 여유도의 해석이 통째로 달라진다.
4. **올바른 사본은 실물에서 이기는가?** makespan은 $c=1$ 을 고르지만 무효 실측은 사본 쪽이 빨랐다. 정직하게 만든 $c=2$ 를 재볼 값어치가 있다.
5. **INT4 경로가 가능한가?** f8 분해가 통했다면 INT4는?
6. **(주최측 미확정)** 채점 서버 URL·인증, 제출 형식·커맨드, 제출 마감·최대 제출 횟수.

11. 참고문헌

하드웨어 · 틀체인 (1차 자료)

- **TCP: A Tensor Contraction Processor for AI Workloads** — ISCA 2024, FuriosaAI. [슬라이드](#) · [발표 영상](#)
- **FuriosaAI RNGD: A Tensor Contraction Processor** — IEEE Micro (Hot Chips 2024 특집). [PDF](#)
- **FuriosaAI RNGD 스펙** — [developer.furiosa.ai](#) (BF16 256 / FP8 512 TFLOPS, INT8 512 / INT4 1024 TOPS, HBM3 48GB @ 1.5TB/s, SRAM 256MB, 5nm, 1.0GHz, 150W)
- **Programming Tensor Contraction Processors** — [furiosa-opt book](#). [Quick Start](#) (슬라이드당 DM 512 KB), [Schedule](#), [Diagnosis](#), [Memory Performance](#), [Kernel Optimizer](#), [Schedule Viewer](#)
- **furiosa_opt_std API** — [docs.rs](#) · [prelude](#) · [contraction](#) · [vector](#)
- **FuriosaAI Arena** — [arena.furiosa.ai](#) · [furiosa-arena-cli](#)

개념 배경 (§2를 더 파고 싶을 때)

- Williams, Waterman & Patterson. **Roofline: An Insightful Visual Performance Model for Multicore Architectures**. CACM 2009 — §2.6의 산술 강도·메모리 바운드 개념의 출처.
- **Einstein summation / tensor contraction** — NumPy [einsum](#) 문서가 §2.2를 손으로 만져보기에 제일 좋다.

모델 아키텍처

- Gemma Team. **Gemma 3 Technical Report**. [arXiv:2503.19786](#) — local/global 교대 sliding-window 어텐션, 128K+ 컨텍스트, 멀티모달. 이 저장소 40+8 레이어 구조의 직계 조상.
- Zhang & Sennrich. **Root Mean Square Layer Normalization**. [arXiv:1910.07467](#) — [shared/rmsnorm.rs](#) 가 구현하는 것.
- Shazeer. **GLU Variants Improve Transformer**. [arXiv:2002.05202](#) — [shared/mlp.rs](#) 의 GeGLU.
- Su et al. **RoFormer: Enhanced Transformer with Rotary Position Embedding**. [arXiv:2104.09864](#) — [sliding/rope.rs](#) ($\theta=10,000$), [full/rope.rs](#) ($\theta=1,000,000$).

- Ainslie et al. **GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.** [arXiv:2305.13245](#) — `Ns=8` KV 헤드에 `Gs=2` 쿼리가 붙는 구조.
- Milakov & Gimelshein. **Online normalizer calculation for softmax.** [arXiv:1805.02867](#) — `full/attention.rs` 의 페이지별 online softmax. Stage 2용.

양자화 (FFN의 NVFP4 경로)

- Elangovan et al. **LO-BCQ: Block Clustered Quantization for 4-bit (W4A4) LLM Inference.** [arXiv:2502.05376](#)
- Dettmers & Zettlemoyer. **The case for 4-bit precision: k-bit Inference Scaling Laws.** [arXiv:2212.09720](#)
- Blumenberg et al. **Improving Block-Wise LLM Quantization by 4-bit Block-Wise Optimal Float (BOF4).** [arXiv:2505.06653](#)

배경 (Stage 2 대비)

- Wang et al. **RATTENTION: Towards the Minimal Sliding Window Size in Local-Global Attention Models.** [arXiv:2506.15545](#)
- Cabannes et al. **Short window attention enables long-term memorization.** [arXiv:2509.24552](#)
- Alexandridis et al. **FLASH-D: FlashAttention with Hidden Softmax Division.** [arXiv:2505.14201](#)
- Chen et al. **INT-FlashAttention: Enabling Flash Attention for INT8 Quantization.** [arXiv:2409.16997](#)

저장소 내부 문서

[RULES.md](#) (최상위 규칙) · [RESULTS.md](#) (실험 기록) · [SOTA.md](#) (신기록 서사) · [README.md](#) (대회 규정 원문) · [OPTIMIZATION.md](#) (최적화 워크플로) · [ARCHITECTURE.md](#) (저장소 지도) · [COMPETITION_INFO.md](#) (공지 원문)

이 보고서의 성능 수치는 RNGD 실측(Arena job 15360, 3/3 PASS)이다. makespan은 \$7.4와 \$9.3에서 실측과의 비교용으로만 등장한다. 최신 실험 기록은 [RESULTS.md](#), SOTA 서사는 [SOTA.md](#)에 있다.